

```
=====
ConferenceConnect ? Innovation Features
Applied Databases Project (25-26-8637)
Student: Gustavo Goes
=====
```

OVERVIEW

Four innovation features were added to the base application, each extending the system in a meaningful and technically distinct way. Three features are accessible from the main menu under options 7, 8, and 9. A fourth runs automatically at startup before any database connection is made.

No additional packages are required. All innovations use only mysql-connector-python, neo4j, and getpass (Python standard library ? no installation needed).

----- INNOVATION 4 ? Startup: Secure Credential Prompt -----

WHAT IT DOES:

Instead of hardcoding database credentials in the source code, the application prompts the user to enter their MySQL and Neo4j credentials interactively at startup, before any connection is attempted. Passwords are hidden while being typed using Python's built-in getpass module.

WHY IT ADDS VALUE:

Hardcoding credentials in source code is a well-known security vulnerability (OWASP A07 ? Identification and Authentication Failures). If the repository is public, any committed password is immediately exposed. This feature also makes the application portable: any user with a local MySQL and Neo4j installation can run it without modifying the source code.

HOW IT WORKS:

1. At launch, a prompt_credentials() function runs before any database connection is made.
2. The user is shown a clear setup banner explaining what is needed.
3. They enter their MySQL username (default: root) and password, then their Neo4j username (default: neo4j) and password.
4. Passwords are collected using getpass.getpass(), which suppresses on-screen echo so the password is never visible in the terminal.
5. The credentials are passed to connect_mysql() and connect_neo4j() as arguments ? nothing is stored as a global variable or written to disk.

KEY TECHNICAL POINTS:

- getpass is part of the Python standard library ? no additional installation required.
- Pressing Enter on the username fields accepts the most common defaults (root / neo4j) to minimise friction.
- No credentials are hardcoded or committed to the repository at any point.
- The design follows the principle of least surprise: the prompt appears once at startup and the rest of the application behaves identically to before.

INNOVATION 1 ? Option 7: Delete Attendee

WHAT IT DOES:

Allows the user to permanently remove an attendee from the system. The deletion is performed across both databases:

- MySQL: removes all registrations for that attendee, then removes the attendee record itself.
- Neo4j: removes the Attendee node and all its CONNECTED_TO relationships (DETACH DELETE).

WHY IT ADDS VALUE:

The base spec only implements Create (Option 3) for attendees. A real conference system must also support removal ? for example, when an attendee cancels. This completes the full CRUD cycle (Create, Read, Update, Delete) for the attendee entity.

HOW IT WORKS:

1. Shows the list of current attendees to help the user choose the correct ID.
2. Looks up the attendee name in MySQL.
3. Asks the user to confirm with "yes" before deleting.
4. Deletes registrations first (foreign key constraint), then the attendee row, then commits.
5. Removes the matching Neo4j node using DETACH DELETE, which also removes all relationships automatically.

KEY TECHNICAL POINTS:

- Registrations are deleted before the attendee to respect the foreign key constraint.
- Both databases are kept in sync ? the node is removed from Neo4j even if the attendee had no connections.
- A confirmation step prevents accidental deletion.

INNOVATION 2 ? Option 8: Most Connected Attendees

WHAT IT DOES:

Queries the Neo4j graph database to find which attendees have the most connections in the network, ranked from most to least connected. Displays the top 10.

WHY IT ADDS VALUE:

This is a graph analytics feature ? something only a graph database like Neo4j can do efficiently. It reveals which attendees are the best networkers at the conference, which has real business value (e.g., identifying key influencers).

HOW IT WORKS:

1. Runs a Cypher query that counts how many CONNECTED_TO relationships each Attendee node has (in either direction).
2. Orders results by connection count descending.
3. For each result, looks up the attendee name in MySQL.
4. Displays a ranked table: Rank | ID | Name | Connections.

KEY TECHNICAL POINTS:

- Uses undirected relationship matching (-[:CONNECTED_TO]-) so direction does not affect the count.
- Combines Neo4j (graph degree counting) with MySQL (name lookup) in a single feature ? demonstrating cross-database integration.
- LIMIT 10 keeps output concise regardless of network size.

----- INNOVATION 3 ? Option 9: Search Attendee by Name -----

WHAT IT DOES:

Allows the user to search for one or more attendees by entering a full or partial name. Returns matching attendees with their ID, date of birth, gender, and company name.

WHY IT ADDS VALUE:

The base spec has no way to look up an attendee by name. Without this, a user must already know the attendee ID to use Options 4 or 5. This feature bridges that gap and makes the application significantly more user-friendly.

HOW IT WORKS:

1. Asks the user to enter a name or partial name.
2. Runs a parameterised SQL query using LIKE with wildcards (%input%) against the attendee table.
3. JOINS with the company table to show the company name instead of a raw company ID.
4. Displays all matches in a formatted table.

KEY TECHNICAL POINTS:

- Uses a parameterised query (%s) to prevent SQL injection.
- The LIKE wildcard works on partial input ? for example, entering "mur" would find "Ava Murphy".
- JOIN with company ensures output is human-readable.

HELPER FEATURE ? Contextual Reference Data Display

WHAT IT DOES:

Whenever the user is asked to enter a Company ID or an Attendee ID (in Options 3, 4, 5, 7), the application automatically displays the current list of valid companies or attendees before asking for input.

WHY IT ADDS VALUE:

Without this, the user has no way to know which IDs are valid without switching to another menu option first. This reduces input errors and improves the overall user experience significantly.

HOW IT WORKS:

Two helper functions ? `show_companies()` and `show_attendees()` ? query MySQL and print a formatted reference table. They are called at the start of the relevant options before any `input()` prompt.

INSTALL INSTRUCTIONS

No additional packages are required beyond what was already installed for the base application:

```
pip install mysql-connector-python
pip install neo4j
```

To run the application:

```
python project/main.py
```

=====